Polymorphism in Java

30 March 2024 18:19

What is Polymorphism?
In Java we can perform a **single action in different ways**.

Runtime Polymorphism or Dynamic method dispatch is a process in which a call to an overridden method to resolved at runtime rather than compile time.

In this process, an overridden methods is called through the reference variable of a superclass. The determination of the method to be called is based on the object referred to by the reference variable.

Upcasting
If a reference variable of Parent class refers to the object of child class, it is known as Upcasting. For example:

```
public class A
{
   private int x;
   A(){
   }
}
public class B extends A
{
}

public class T
{
   public void test(){
      A hold = new B(); //Upcasting
   }
}
```

object of child class.

reference variable of parent class
refer to the object of child class.

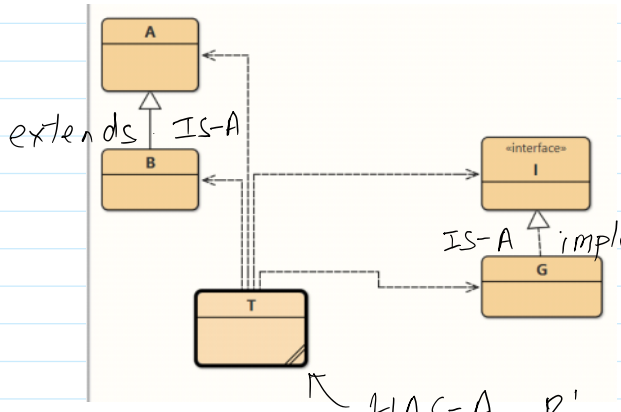
For upcasting, we can use the reference variable of class type or interface type.

```
public interface I
{
   int sampleMethod(int y);
}

public class G implements I
{
   @Override
   public int sampleMethod(int x){
      return x;
   }
}

public class T
{
   public void test(){
```

```
    A hold = new B(); //Upcasting
  }

  public void testI(){
    I ref = new G(); //upcasting using interface reference variable
  }
}
```

Interface type reference variable.

child class		parent class/interface.
B	IS-A	A (class)
G	IS-A	I (interface)



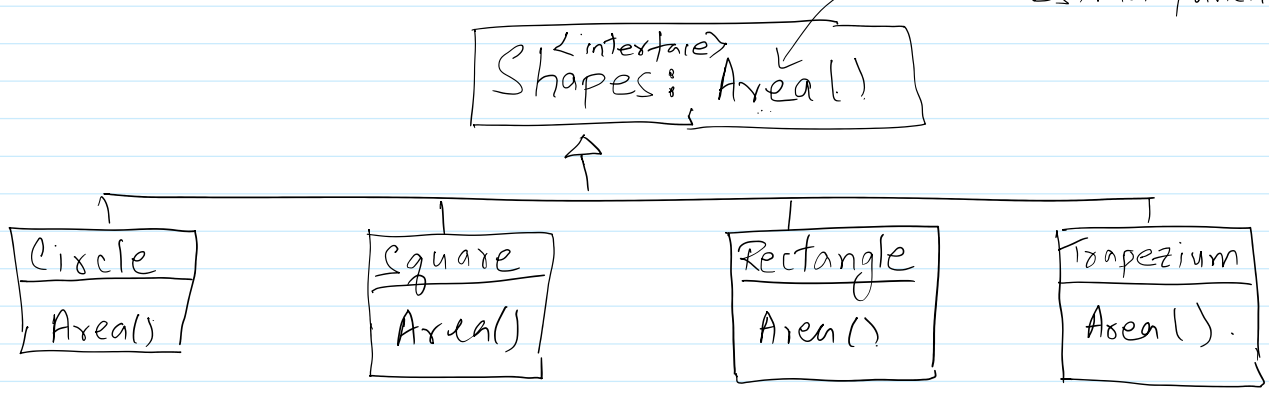
HAS-A B's object

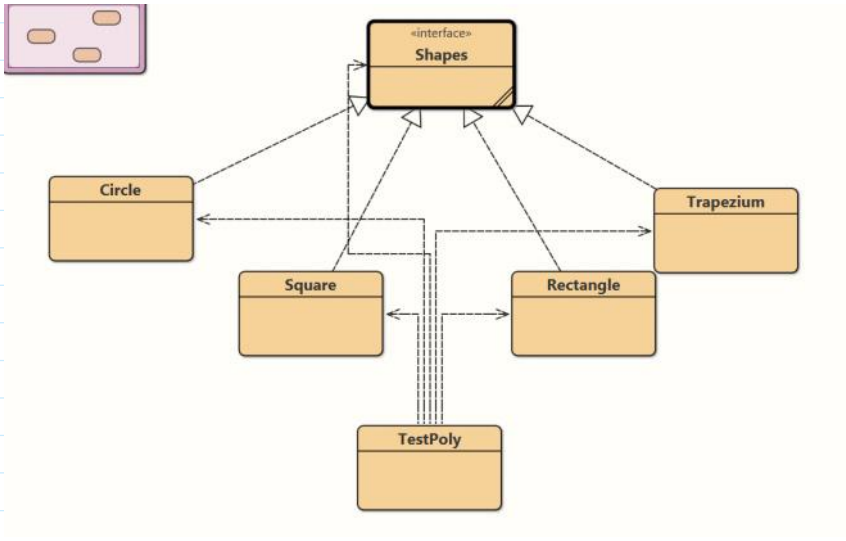
HAS-A G's object

HAS-A A's reference

HAS-A I's reference.

is an abstract function.





Unified language

UML-class diagram

Modeling

which represents the relationship between classes and interface.

```
public interface Shapes
{
   double Area();
}

import java.util.*;
public class Circle implements Shapes
{
   private double radius;
   @Override
   public double Area(){
      Scanner sc = new Scanner(System.in);
      System.out.print("\nEnter the radius of the circle: ");
      radius = sc.nextDouble();
      return Math.PI * radius * radius;
   }
}

import java.util.*;
public class Square implements Shapes
{
   private double side;
   @Override
   public double Area(){
      Scanner sc = new Scanner(System.in);
      System.out.print("\nEnter the length of the side of a square: ");
      side = sc.nextDouble();
      return side * side;
   }
}


import java.util.*;
public class Rectangle implements Shapes
{
   private double length, breadth;
   @Override
   public double Area(){
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter the length of the rectangle: ");
        length = sc.nextDouble();
        System.out.print("\nEnter the breadth of the rectangle: ");
        breadth = sc.nextDouble();
        return length * breadth;
    }
}


import java.util.*;
public class Trapezium implements Shapes
{
    private double side1, side2, height;
    @Override
    public double Area(){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter the side1 of the trapezium: ");
        side1= sc.nextDouble();
        System.out.print("\nEnter the side2 of the trapezium: ");
        side2 = sc.nextDouble();
        System.out.print("\nEnter the height of the trapezium: ");
        height = sc.nextDouble();
        return 0.5 * (side1 + side2)*height;
    }
}

public class TestPoly
{
    public static void main(String args[]){
        System.out.print("\f");
        //array of reference variables
        Shapes shape[] = new Shapes[4];
        shape[0] = new Circle(); //Upcasting
        shape[1] = new Rectangle();
        shape[2] = new Square();
        shape[3] = new Trapezium();
        int i;
        for(i = 0; i < shape.length; i++){
            //decided on the runctime which Area() method will be called
            //from which class.
            double ar = shape[i].Area();
            System.out.print("\n"+shape[i].getClass().getName()+": "+ar+" sq. units");
        }
    }
}
```

this method is called during runtime

argument for getClass()

reference variable of shapes keeping the reference of which child class (decided at runtime).

return me object of class Class.

this class inherits Object class.

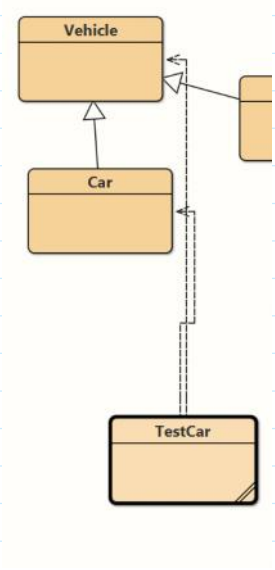
getName()

String class.

→ String class.
→ Garbage collection.
→ Inheritance
→ Threads.
→ Applet program
or
swing

instanceof Operator

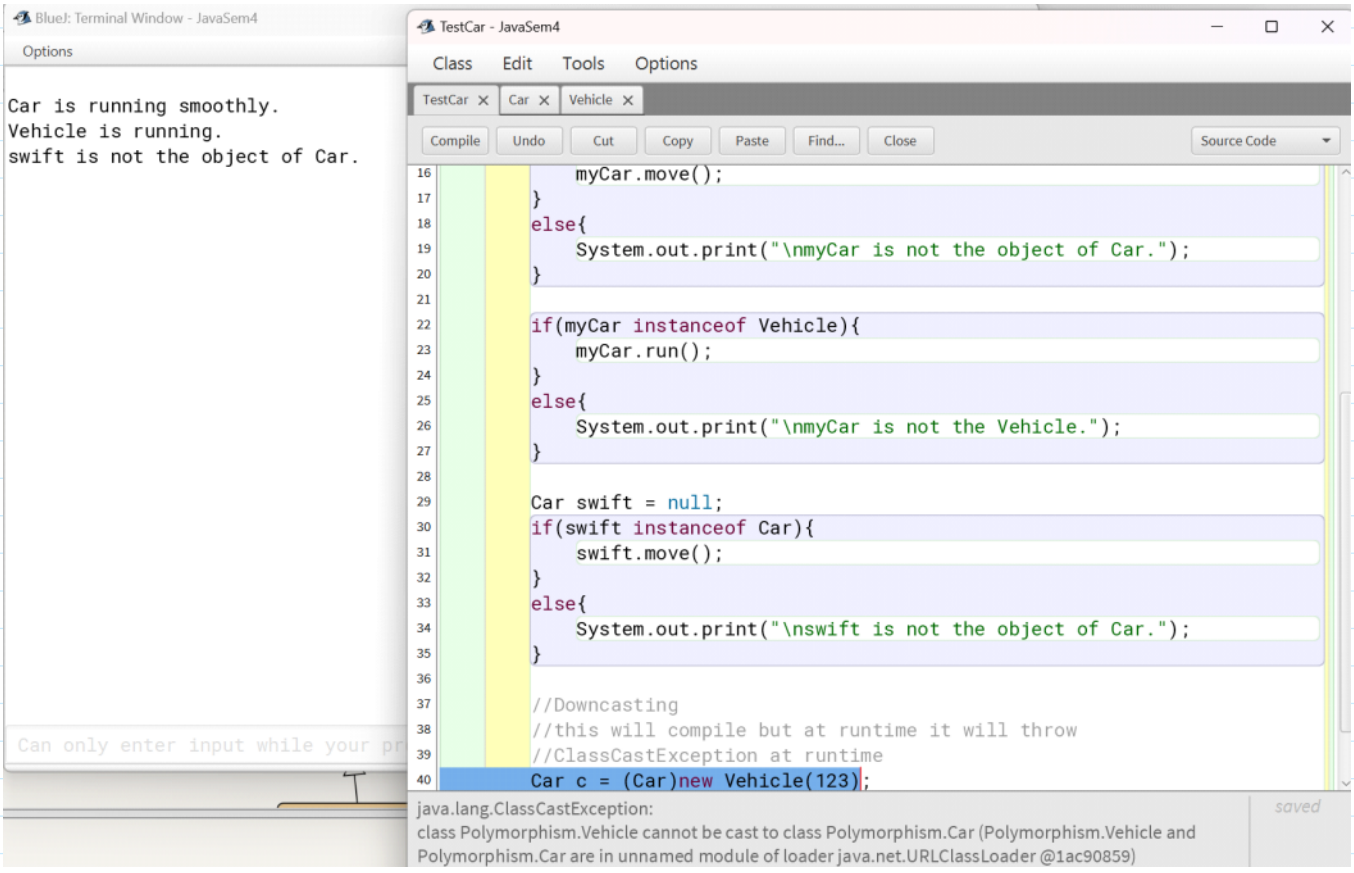
The Java **instanceof** operator is used to check or test whether the object is an instance of the specified type or class(or subclass or interface).
The **instanceof** in Java. Is also known as **type comparison operator** because it compares the instance with type. It returns either true or false. If we apply the **instanceof** operator, with any variable that has null value, it returns false.




```
public class Vehicle
{
   private int engineNum;
   public Vehicle(int en){
      this.engineNum = en;
   }

   final public void run(){
      System.out.print("\nVehicle is running.");
   }

   public Vehicle getVehicle(){
      return new Vehicle(engineNum); //returning the object of Vehicle.
   }
}
public class Car extends Vehicle
{
   /**
    * @param engineNum we need to provide engine number
    */
   Car(int engineNum){
      super(engineNum);
   }

   public void move(){
      //super.run();    //reusing the parent class code by calling its method.
      System.out.print("\nCar is running smoothly.");
   }

   @Override
   public Car getVehicle(){
      return new Car(12345); //returning the object of Vehicle.
   }
}
```

```
public class TestCar
{
   public static void main(String args[]){
      System.out.print("\f");
      Car myCar = new Car(123);
      if(myCar instanceof Car){
         myCar.move();
      }
      else{
         System.out.print("\nmyCar is not the object of Car.");
      }

      if(myCar instanceof Vehicle){
         myCar.run();
      }
      else{
         System.out.print("\nmyCar is not the Vehicle.");
      }

      Car swift = null;
      if(swift instanceof Car){
         swift.move();
      }
      else{
         System.out.print("\nswift is not the object of Car.");
      }
   }
}
```

Possibility of downcasting with instance of:



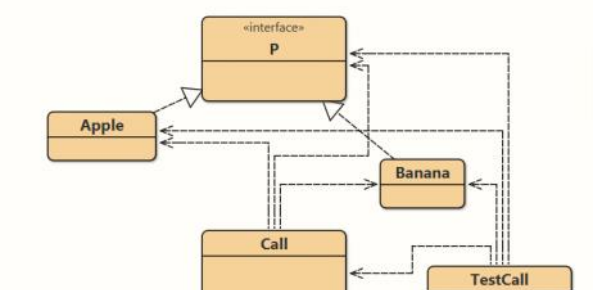
Solution of the above problem:

```
package Polymorphism;


/**
 * Write a description of class TestCar here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestCar
{
   public static void main(String args[]){
      System.out.print("\f");
      Car myCar = new Car(123);
      if(myCar instanceof Car){
         myCar.move();
      }
      else{
         System.out.print("\nmyCar is not the object of Car.");
      }

      if(myCar instanceof Vehicle){
         myCar.run();
      }
      else{
         System.out.print("\nmyCar is not the Vehicle.");
      }
```

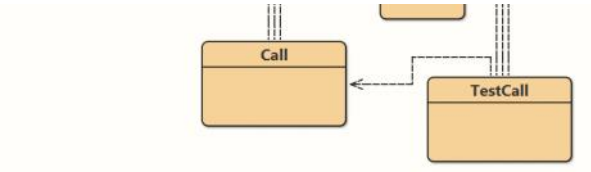
```
        Car swift = null;
        if(swift instanceof Car){
            swift.move();
        }
        else{
            System.out.print("\nswift is not the object of Car.");
        }

        //Downcasting
        //this will compile but at runtime it will throw
        //ClassCastException at runtime
        //Car c = (Car)new Vehicle(123);
        Vehicle v = new Car(123);
        method(v);
    }

    public static void method(Vehicle v){
        if (v instanceof Car){
            Car c = (Car)v; //downcasting
            System.out.print("\nDowncasting is performed.");
        }
        else{
            System.out.print("\nUnable to do downcasting.");
        }
    }
}
```

Another example using interface:

```
public interface P
{
}
public class Apple implements P
{
    public void methodOfApple(){
        System.out.print("\nMethod in Apple");
    }
}
public class Banana implements P
{
    public void bananaMethod(){
        System.out.print("\nThis is banana method.");
    }
}
public class Call
{
    public void invoke(P p){ //upcasting
        if(p instanceof Apple){
            Apple a = (Apple)p;//downcasting
            a.methodOfApple();
        }

        if(p instanceof Banana){
            Banana a = (Banana)p;//downcasting
            a.bananaMethod();
        }
```



```
        if(p instanceof Banana){
            Banana a = (Banana)p;//downcasting
            a.bananaMethod();
        }
    }
}
public class TestCall
{
    public static void main(String args[]){
        System.out.print("\f");
        P p = new Banana();
        Call c = new Call();
        c.invoke(p);

        p = new Apple();
        c.invoke(p);
    }
}
```



Relationship diagram between classes and interface.